FoveaWAM: Role-Aware 4D World–Action Modeling for Dynamic Manipulation Supplementary Material

Anonymous CVPR submission

A. Method and Training Details

A.1. Error Decomposition

This subsection bounds the error of a policy's estimate of the target pose at the execution time τ^{a} of a command (Eq. (S5)), derives the rotation and context terms behind the task groups of Appendix B.1, and shows how the error scales with the lead and the motion. Let a policy act, explicitly or implicitly, on an estimate of the target pose at τ^{a} with position error e and rotation error θ, so that a contact point at lever arm r is off by at most $e + 2r\sin(\theta/2)$. Writing the target's motion over the lead $\Delta = \tau^{\mathrm{a}} - \tau_t$ as a constant velocity $\mathbf{v}$ plus a residual (acceleration, rolling, impacts), and the lead that the policy assumes as $\hat{\Delta}$, gives

$$e \le e_{\mathrm{loc}} + \hat{\Delta}\, e_v + \|\mathbf{v}\|\, e_\tau + e_{\mathrm{dyn}}, \tag{S1}$$

with the errors e_{loc} of the current position, e_v of the velocity, $e_\tau = |\hat{\Delta} - \Delta|$ of the timing, and e_{dyn} of the residual motion.

Translation. Write the true target position at the execution time as $\mathbf{p}(\tau^{\mathrm{a}}) = \mathbf{p}(\tau_t) + \mathbf{v}\Delta + \boldsymbol{\rho}$, with the velocity $\mathbf{v}$ at τ_t and the remaining motion $\boldsymbol{\rho}$, and the policy's estimate as $\hat{\mathbf{p}}(\tau^{\mathrm{a}}) = \hat{\mathbf{p}}(\tau_t) + \hat{\mathbf{v}}\hat{\Delta} + \hat{\boldsymbol{\rho}}$. Subtracting and adding $\mathbf{v}\hat{\Delta}$ gives

$$\begin{aligned}\hat{\mathbf{p}}(\tau^{\mathrm{a}}) - \mathbf{p}(\tau^{\mathrm{a}}) = \big[\hat{\mathbf{p}}(\tau_t) - \mathbf{p}(\tau_t)\big] + (\hat{\mathbf{v}} - \mathbf{v})\hat{\Delta} \\ + \mathbf{v}(\hat{\Delta} - \Delta) + (\hat{\boldsymbol{\rho}} - \boldsymbol{\rho}),\end{aligned} \tag{S2}$$

and the triangle inequality yields Eq. (S1) with $e_v = \|\hat{\mathbf{v}} - \mathbf{v}\|$ and $e_{\mathrm{dyn}} = \|\hat{\boldsymbol{\rho}} - \boldsymbol{\rho}\|$. The bound holds for any split of the estimate into $\hat{\mathbf{v}}\hat{\Delta}$ and $\hat{\boldsymbol{\rho}}$, and e_{dyn} absorbs the higher-order part of the timing error. The residual $\boldsymbol{\rho}$ is the motion that constant-velocity extrapolation misses. Under a constant acceleration $\mathbf{a}$, $\boldsymbol{\rho} = \frac{1}{2}\mathbf{a}\Delta^2$, which an acceleration estimate from the history reduces; impacts, rolling contact, and friction changes leave a remainder that only a model of the dynamics can reduce.

Rotation and the contact point. Write the true orientation as $R(\tau^{\mathrm{a}}) = \mathrm{Exp}(\boldsymbol{\omega}\Delta + \boldsymbol{\eta})R(\tau_t)$, with the angular velocity $\boldsymbol{\omega}$ and the remaining rotation $\boldsymbol{\eta}$, and its estimate likewise. The rotation angle is bi-invariant, and Exp is 1-Lipschitz from $\mathbb{R}^3$ to SO(3), so for general rotations

$$\begin{aligned}\theta &= \big\| \mathrm{Log}(\hat{R}(\tau^{\mathrm{a}})R(\tau^{\mathrm{a}})^\top)^\vee \big\| \\ &\le \theta_{\mathrm{loc}} + \|\hat{\boldsymbol{\omega}}\hat{\Delta} + \hat{\boldsymbol{\eta}} - \boldsymbol{\omega}\Delta - \boldsymbol{\eta}\| \\ &\le \theta_{\mathrm{loc}} + \hat{\Delta}\, e_\omega + \|\boldsymbol{\omega}\|\, e_\tau + e_\eta,\end{aligned} \tag{S3}$$

with the current orientation error θ_{loc}, $e_\omega = \|\hat{\boldsymbol{\omega}} - \boldsymbol{\omega}\|$, and $e_\eta = \|\hat{\boldsymbol{\eta}} - \boldsymbol{\eta}\|$. A contact point at $\mathbf{c}$ in the object frame, with $\|\mathbf{c}\| = r$, is off by $(\hat{\mathbf{p}} - \mathbf{p}) + (\hat{R}R^\top - I)R\mathbf{c}$ at τ^{a}, and the operator norm of $\hat{R}R^\top - I$ is $2\sin(\theta/2)$, so its error is

$$e_{\mathrm{c}} \le e + 2r\sin(\theta/2) \le e + r\theta, \tag{S4}$$

with $e = \|\hat{\mathbf{p}}(\tau^{\mathrm{a}}) - \mathbf{p}(\tau^{\mathrm{a}})\|$. A grasp or placement constrains several points spread over the body, such as the two finger contacts or the arms of a T-shaped part, so no single tracked point removes the term $r\theta$: the orientation error enters the contact scaled by the lever arm, and the target needs its full SE(3) state.

Context. Let m be the largest margin that a feasible path can keep between the robot and the carried object and the context it passes, measured to the predicted positions of the context at τ^{a}. If the executed path keeps this margin, a collision requires a context error of at least m, so a coarse estimate of the surroundings suffices when $m \gg \varepsilon$; the no-scene-flow control of Sec. 4.3 tests this coarse supervision directly. A driver, such as a conveyor, a ramp, or a striker, acts on the contact through the motion it imparts to the target, which enters Eq. (S1) through the target's own terms. The SE(3) history and forecast represent this motion precisely, so the driver needs no precise state of its own.

A.2. Architecture

Table S1 lists the configuration of every module, in the order of the data flow described below.

Scene tokens. The frozen MAE ViT-B/16 [7] encodes the agent-view RGB into 196 patch tokens; we selected it on development scenes (Appendix E.4). Each patch token adds a

Table S1. Architecture of FoveaWAM. Heads are skipped at deployment.

Module	Configuration
Visual encoder	frozen MAE ViT-B/16, 224×224 input, 196 patch tokens
Target-state tokenizer	$28 \to 1024 \to 1024$ GELU MLP, additive relative-time embedding
Robot-state tokenizer	linear, $7 \to 1024$
Tokens	212 scene (196 patch, 8 target-state, 8 robot), 16 summary, 8 action
Backbone	24 layers, width 1024, 16 heads, shared attention, SiLU FFN 4096 per stream, 504.0M parameters
Pose head	linear, mean summary state $\to 8 \times 6$
Flow head	linear per patch token, $8 \times (3{+}1)$
Appearance heads	linear per patch token: depth and moving-surface RGB as 16×16 pixels, 1024-d DINOv3 ViT-L/16 features
Target reconstruction	$1024 \to 256 \to 3072$ GELU MLP with sigmoid, 32×32 RGB
Action stream	flow matching, adaptive layer norm on the flow time, 5 Euler steps

learned view embedding, shared by all patches in our single-view setting, and linear embeddings of its mean valid metric depth and of the base-frame position of its back-projected center pixel. A patch whose center pixel has no valid depth keeps its visual token but receives no position embedding. These positions also drive 3D rotary embeddings of the patch queries and keys, which no other token receives, and the same center points are the source points of the sparse 3D flow. Past images never enter the model.

State tokens. The 8 target-state tokens sample the offsets $-350, -300, \ldots, 0$ ms. Each is a 25-dimensional descriptor $(\mathbf{p}, \log R, \Delta\mathbf{p}, \log \Delta R, \Delta t, \mathbf{v}, \boldsymbol{\omega}, \mathbf{a}, \boldsymbol{\alpha})$ with dimensions $(3, 3, 3, 3, 1, 3, 3, 3, 3)$: slots 0–6 describe the seven intervals between consecutive offsets, with velocities from interval displacements over the elapsed time and accelerations from adjacent velocities over the spacing of the interval centers, and slot 7 holds the current absolute pose in the base frame. We standardize each dimension with training statistics, zero invalid dimensions, and append three validity flags (pose, interval, and acceleration); a slot without any valid feature is masked out of attention. On the real robot, the tracker updates the pose at every control-period boundary (Appendix D.2), so the history keeps the same 50 ms spacing. The robot tokens hold the end-effector position, rotation vector, and measured gripper opening at the same offsets, and 16 learned embeddings form the summary tokens. The policy has no language token.

Backbone and mask. Each layer applies one attention over all tokens with query, key, value, and output projections shared by both streams, followed by separate SiLU feed-forward networks for the world and the action tokens. The mask of Sec. 3.4 gives a blocked edge the logit $-\infty$ in every layer and head, in training and at deployment, so the world states are computed once per query and reused across the Euler steps.

Forecast and auxiliary heads. All heads are readouts of the final world states, trained jointly with the action stream and skipped at deployment; no token reads their outputs. The pose head reads the mean summary state and is trained with $\mathcal{L}_P$ (Eq. 3). The flow head gives each patch point's displacement and visibility logit at every horizon, trained with $\mathcal{L}_F$ (Eq. 2) and the scale $s = 0.7154605$ m; the visibility loss has weight 1 in simulation, and points that are valid but invisible still supervise their displacement. The appearance heads predict at every horizon the metric depth with a squared error on valid pixels, the RGB of surfaces that move by more than 1 mm, including the robot, with a squared error, and the DINOv3 features [13] with a squared error after channel-wise layer normalization. The target-reconstruction head recovers the current target's RGB region from the mean summary state with an absolute error. These four losses form $\mathcal{L}_{\text{aux}}$.

Action stream. Each of the 8 action tokens embeds a seven-dimensional command, a base-frame translation increment, a left-relative rotation vector, and a gripper command; the increments refer to the measured end-effector pose at application. At deployment, we integrate 5 Euler steps, reverse the normalization, decode the rotation vectors, and clip each translation axis to 0.10 m and each rotation axis to 0.5 rad; the gripper changes state only when its command leaves a deadband of 0.5 in the ball-capture tasks and of 0.1 in the others. Appendix B.3 specifies how the chunks are executed.

A.3. Training on ConMoto

Each control step of a demonstration anchors one window of 8 query positions with their own causal histories and no attention across positions, which gives 1,324,374 windows from the 11,200 demonstrations; 350 further episodes form the validation split. For the flow labels, each patch-center pixel is back-projected with the rendered depth and attached to the local frame of the MuJoCo geometry rendered at it; the ground-truth rigid transforms of that geometry carry the same surface point to the future horizons, which gives its cumulative displacement in the base frame. A future point is visible if it lies in front of the camera and inside the image and its projected pixel renders the same geometry at a depth within $\max(1\,\text{mm}, 0.001z)$ of the point's depth z. Command j of the chunk for query t is labeled with

the demonstration command issued at $\tau_t + j\Delta_a$, since d_{task} applies only at execution (Eq. (S5)), and commands are normalized with the mean and standard deviation of the valid commands of the training split. Each query position draws 8 noise–time pairs for its chunk and averages their losses, which reduces the variance of the action gradient without enlarging the batch, and each loss is normalized by its count of valid labels in the global batch. The DINOv3 features of future frames are cached once per episode. We train for at most 20 epochs and keep the checkpoint with the lowest loss on the validation split. Table S2 lists the recipe.

Table S2. Training hyperparameters on ConMoto.

Item	Value
Optimizer	AdamW, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, weight decay 10^{-4}
Learning rate	2×10^{-4}, 5,174 warm-up updates from 0, cosine decay to 0
Batch, schedule	global batch 512 windows, at most 20 epochs, lowest-validation-loss checkpoint
Loss weights	$\lambda_F = \lambda_P = \lambda_R = 0.1$, $\lambda_{\text{aux}} = 0.1$ per target
Action noise	8 noise–time pairs per query position
Precision	trainable weights FP32, frozen encoder BF16, gradient clipping at 0.1
Inference	FP32, world states cached within a query
Hardware	[N] RTX PRO 6000 GPUs, [X] hours per run

A.4. Pretraining on DROID and Fine-Tuning

Data. The real-robot base trains on DROID [9], recorded at 15 Hz, with the 3D point tracks and camera calibrations that PointWorld released for it [8], matched to the raw episodes by exact agreement of robot states, commands, and camera identity. It uses the first exterior camera with its depth. Of 42,935 matched episodes, 42,087 yield 598,889 anchor windows, 538,740 from 37,874 training and 60,149 from 4,213 validation episodes. Each anchor expands into 8 consecutive query positions; the anchor carries the flow and DINOv3 labels, and the others carry action labels only. All times are capture timestamps, and the flow and DINOv3 targets keep PointWorld's record times. We keep the 96.88% of action slots with an issued command, without a filter on speed, so that the base is not biased toward slow motion.

Objectives. DROID provides no object poses, so the base has no target-state tokens or pose head. It reads the current RGB-D frame and the robot history (204 scene tokens) and trains the action loss and the sparse 3D flow with weight 1 and the future DINOv3 features with weight 0.1; PointWorld records only visible positions, so the visibility loss is off. Most DROID objects move only when the robot moves them, so the base mainly learns metric geometry and the motion caused by the robot's own actions; the exogenous motion of our tasks and the target pose enter during fine-tuning.

Flow labels. Each patch of the 14×14 grid follows one PointWorld track: among the tracks that start inside the patch, are initially visible, and have a valid depth, the one with the largest image motion consistent with its 3D displacement, and otherwise the one nearest to the patch center, so that moving objects missed by a fixed center point are covered. The label is the track's displacement from its start; 47.05% of the point–horizon slots carry a label.

Recipe and fine-tuning. The base has 48 layers at width 1024 (1.02B trainable parameters) and trains for 20 epochs (21,060 updates, 2,106 warm-up updates) at a global batch of 512 on 64 GPUs, with the optimizer and precision of Table S2. Each real-robot policy starts from the base and is fine-tuned on the demonstrations of its task [updates, learning rate, and batch size]; the target-state tokens and the heads that the base lacks are freshly initialized. No real-robot policy uses simulation weights, and the base sees no simulation data.

B. ConMoto and Evaluation Protocol

B.1. Benchmark Design

ConMoto comprises fourteen tasks in five families (Fig. 3; Sec. 4.1), which together cover translation, rotation, temporary occlusion, constrained passage, and abrupt changes of the target's motion. We will release the tasks, the accepted scenarios, the success checks, and the demonstrations.

Environment. The tasks run in MuJoCo 3.11.0 and robosuite 1.5.2 [14, 19] with a Franka Panda arm and a parallel gripper, driven by robosuite's operational-space pose controller at a control period of 50 ms and a physics step of 2 ms. A fixed third-person agent-view camera at $(0.68, -0.58, 1.62)$ m with a 40° vertical field of view renders 224×224 RGB-D images. An action is an end-effector increment in the robot base frame, a translation and a rotation vector, together with a gripper command; the shared action adapter clips each command to 0.10 m per translation axis and 0.5 rad per rotation-vector component.

Success and termination. Table S3 summarizes the success condition of each task. We check them at every physics step, an interrupted continuous condition resets its counter, and an episode ends at its first terminal event: success, a failure such as unsafe contact, leaving the workspace, or a drop, or the time limit.

Table S3. **The fourteen ConMoto tasks.** Target: the body whose pose the policy receives (Sec. 3.1). Speed: target speed range (capture tasks: at the ramp exit). Every task requires its stages in order without a terminal failure; a grasp requires bilateral fingerpad contact, a lift of at least 0.05 m, and a 0.5 s hold. Limit: episode time limit. Groups of Fig. 5: R, rotation-sensitive; C, context-active (Appendix B.1). The released task configurations contain the exact thresholds and scenario ranges.

Task (code)	Target	Speed (m/s)	Success	Limit (s)	Group
Harmonic grasp (grasp)	block	0.03–0.20	Grasp the block moving harmonically and yawing, lift, and hold.	8	R
Conveyor grasp (t1_belt)	part	0.04–0.10	Grasp the part on the conveyor, lift, and hold.	8	C
Translating grasp (t1_xyz)	part	0.04–0.15	Grasp the translating part, lift, and hold.	8	–
Rotating grasp (t1_so3)	part	0.04–0.15	Grasp the translating and rotating part ($\|\omega\| \leq 1.2$ rad/s), lift, and hold.	8	R
Occluded grasp (t1_occ)	part	0.04–0.10	Grasp the part on the conveyor, which a canopy hides for 0.5–3.0 s, without unsafe contact with the canopy; lift and hold.	8	C
Moving receptacle (t2_trans)	receptacle	0.03–0.06	Place the held object fully inside the moving receptacle and release it to rest.	10	–
Rotating receptacle (t2_se3)	T-shaped receptacle	0.03–0.06	Place the held T-shaped part fully inside the translating and rotating T-shaped receptacle and release it to rest.	10	R
Wire passage (t3_phase)	wire entry	[X]	Align the held wand's loop with the rotating wire within 30 mm, then pass it along the wire to the end without contact.	21	R, C
Full wire passage (t3_full)	wire entry	[X]	Grasp the wand, complete the passage, exit the wire, and release the wand.	32	R, C
Ball capture (t4_sphere)	ball	[0.4–0.7]	Catch the ball after it leaves the ramp and before it touches the tray, lift, and hold.	6	C
Ball capture on table (t4_sphere_tabletop)	ball	[0.4–0.7]	Grasp the ball while it still rolls on the table after the ramp, lift, and hold.	6	C
Slider capture (t4_slider)	slider	[0.4–0.8]	Grasp the block while it still slides on the table after the ramp, lift, and hold.	6	C
Can capture (t4_can)	can	[0.4–0.7]	Grasp the can while it still rolls on the table after the ramp, before it tips over, lift, and hold.	6	R, C
Post-impact grasp (t5_replan)	part	0.12–0.14	After a striker deflects the part on the conveyor, grasp it along its grasp axis, lift, and hold.	6	R, C

Roles and task groups. Table S3 names the target of each task. A moving receptacle is the target, since its pose decides containment; in the passage tasks, the wire entry, which the wand must match within 30 mm, is the target, and the rest of the wire is context that no contact may touch. The two task groups of Fig. 5 were fixed before the ablations from the terms of Appendix A.1: a task is rotation-sensitive ($r\theta$) if its target rotates during the lead and its orientation matters at contact, which excludes the balls, and context-active if it has an obstacle clearance m or a driver (a conveyor, a ramp, or a striker).

B.2. Scenarios, Demonstrations, and Splits

Scenarios. Each scene samples the target's initial state and motion parameters uniformly within task-specific ranges, and discrete choices with equal probability: besides the speeds of Table S3, harmonic amplitudes of up to 35 mm at 0.25–0.7 Hz, yaw rates of up to 1.2 rad/s, wire rotation rates of 2.5–7.5°/s, and ramp slopes of 5–15° with friction set per object. An occluded scene is accepted only if the target is visible for at least eight initial frames, fully occluded for 0.5–3.0 s, and visible again for at least three frames. We keep the scenes that pass physical-feasibility checks and replay the stored accepted scenes at evaluation.

Demonstrations. The expert is a task-specific closed-loop state machine with privileged simulator state: the target's current pose and linear and angular velocity, the robot state, the task geometry, and true contact, grasp, and stage events. At every control period it, for example, extrapolates the target over a short horizon and tracks, descends, closes, and lifts in the grasping tasks, predicts the interception point of a rolling or flying object under gravity in the capture tasks, and follows the known wire path with phase compensation from the current angle and angular velocity in the passage tasks. Its commands are clipped to 0.05 m and 0.5 rad per axis. A demonstration is accepted if its scene is feasible, the expert succeeds, an independent replay of the recorded actions reproduces the episode, and its path, action changes, and jerk pass numerical quality checks. The training pool of 11,200 demonstrations, 800 per task, is drawn from the accepted ones. We record them at 20 Hz with camera images, depth, robot states, issued commands, and simulator ground truth for target poses and scene geometry; the learned policies receive only the inputs of Sec. 3.

Splits. We select configurations on 1,400 development scenes (100 per task). All ConMoto results use a separate held-out set of 1,400 scenes, generated with the same ranges and new source seeds that appear in no training, validation, or development episode; the first 100 feasible scenes in increasing seed order form each task's set, which all methods share.

B.3. Real-Time Evaluation

Executor. All methods share one executor on the 50 ms control clock. Command j of query t is indexed by its intended issue time $\tau_t + j\Delta_a$; if executed, its execution time (Sec. 3.1) is

$$\tau^{\mathrm{a}}_{t,j} = \tau_t + j\Delta_a + d_{\mathrm{task}}, \tag{S5}$$

where $d_{\mathrm{task}} = 50$ ms for the post-impact grasp, which applies each command 50 ms after it is issued, and $d_{\mathrm{task}} = 0$ for all other tasks. The total latency L_t of a query is its inference latency under the protocol plus any added latency. The executor counts latency in whole control periods, as RTC does [2]: a query hands over its chunk at the first control-period boundary at or after it ends, $k_t = \lceil L_t/\Delta_a \rceil$ periods after capture, and execution continues from command k_t, the one meant for that boundary. There the policy observes again and starts the next query; until then, the executor continues the previous chunk, so a policy replans as often as its latency allows. A method's chunk holds its native number H_m of commands (Table S4), and a chunk with $k_t \geq H_m$ has expired entirely and is dropped. With $L_t = 0$, the next query starts one control period after capture. Before the first chunk of an episode arrives, and whenever a chunk runs out, the controller holds the current arm target and gripper command while physics advances; no command is repeated. Every command of FoveaWAM is executed at most $(H-1)\Delta_a + d_{\mathrm{task}}$ after capture, within its forecast horizons (Sec. 3.3).

Protocols. Under the real-time protocol, physics advances during each query by the method's native latency, drawn per query from its latency distribution measured on one RTX 5090. With paused physics, the executor is unchanged but inference advances no physics time: at zero added latency it replans every 50 ms and issues the first command of each new chunk. This is a no-inference-wait control, not a performance upper bound; its difference from real time includes the resulting change in feedback frequency as well as action age. With equal latency, every method is charged the p95 latency of FoveaWAM; with original executors, each baseline keeps the execution scheme of its paper, such as temporal aggregation for ACT or streaming for DynamicVLA, at its native latency. The added-latency sweep of Table S7 adds 0, 50, 100, 150, or 200 ms after inference under the real-time protocol, and physics advances during it. The reported rollouts run on RTX PRO 6000 and replay the latency measured on RTX 5090; concurrent rollout wall time does not enter the simulated delay.

Latency measurement. We time all methods on one RTX 5090 (32 GB), the GPU of our robot, with exclusive use and batch size 1. Each method runs three rounds of thirty warm-up and 300 measured calls over recorded observations with each method's required causal history, with framework-native synchronization at call boundaries, in the precision of its release; FoveaWAM uses FP32. The initial calibration rotates method order; configuration corrections replace only affected methods' complete 900-call distributions. A timed query starts from the newest observation, with the history already maintained, and ends when its first command is available. It includes all computation that needs the newest frame: preprocessing, image encoding, the online flow updates and motion descriptors of PUMA and DynamicWAM, sampling, and action decoding, and on the robot also tracking. For FoveaWAM, we time the forecast heads separately, although deployment skips them. This time-to-action is reported without rounding; the handover time $\ell_t = k_t\Delta_a$ rounds it up to the control period, and the action age of a command is its execution time minus the capture time. A query misses its deadline when its time-to-action exceeds 50 ms. Appendix C.2 reports the measured values.

B.4. Metrics and Statistics

The task-balanced success rate is

$$\mathrm{SR}(m) = \frac{100}{14}\sum_{k=1}^{14}\frac{1}{n_k}\sum_{i=1}^{n_k} y_{mki}, \tag{S6}$$

for binary outcomes y_{mki} of method m on scene i of task k. The *position error at first contact* e_p is the distance of the end effector from the expert's contact frame on the target. In the two placement tasks, which start with the object held, the contact is that of the held object with the receptacle, and the contact frame is the end-effector pose of the expert's placement. e_p averages over the 13 tasks with a contact, excluding `t3_phase`, and over the episodes that make contact; failures without contact stay in the SR denominator.

Paired intervals. Every method is trained once. For paired 95% intervals, we bootstrap held-out scenes within each task, jointly across methods; for the ablations, we also repeat the comparison without the phase-only wire passage (`t3_phase`, Appendix C.4). These intervals quantify scene uncertainty for the trained weights, and we treat differences whose interval lies within ± 2 points as ties.

B.5. Baselines and Ablation Controls

Common conditions. Each baseline keeps its official architecture, chunk length, initialization, and training recipe

Table S4. **Baseline training.** All methods train on the same 11,200 demonstrations with the official recipe of each method and run under the shared executor of Appendix B.3. Updates × batch: training budget. H: commands per returned chunk. Params: trainable parameters, frozen encoders excluded. DOM ckpt.: the released DynamicVLA checkpoint trained on DOM. OpenWAM-α generates video and actions jointly with video decoding disabled.

Method	Initialization	Updates×batch	H	Params
ACT [18]	scratch	$50{,}000 \times 32$	100	[X]
DP [4]	scratch	$50{,}000 \times 32$	15	[X]
$\pi_{0.5}$ [1]	`pi05_base`	$30{,}000 \times 256$	10	[X]
$\pi_{0.5}$+SE(3)	`pi05_base`	$30{,}000 \times 256$	10	[X]
DynamicVLA [17]	DOM ckpt.	$50{,}000 \times 44$	20	[X]
PUMA [5]	official	[X]×[X]	[X]	[X]
OpenWAM-α [15]	official	$34{,}490 \times 192$	32	[X]
DynamicWAM [11]	official	$40{,}000 \times 128$	16	[X]
DynamicWAM+SE(3)	official	$40{,}000 \times 128$	16	[X]
FoveaWAM	scratch	$51{,}740 \times 512$	8	523.8M

(Table S4), trains once with seed 0 on the same 11,200 demonstrations, whose commands we map to its action space through the same Cartesian controller, and is configured on development scenes. The reported rollouts use the trained agent view and the further histories of Table 1; required missing wrist slots remain masked or black. Their latency budgets instead use separate measurements with a real wrist view in addition to the agent view. Thus the reported SR evaluates single-view policies under two-view deployment budgets, not policies trained with two views. For the seven baseline configurations timed both ways, the agent view alone changes the p95 latency by at most 3.8 ms, less than a tenth of the 50 ms control period: most of them keep the camera slot or fixed canvas of the wrist view without it, so the wrist view mainly replaces its content, and only ACT and DP encode an additional camera. PUMA, OpenWAM with its official acceleration, and DynamicWAM+SE(3) were timed only in their deployment setting. DP uses its author's real-robot DDIM16 sampler, returning 15 commands; the prior DDPM100 results are not used here. OpenWAM's official cache/compile configuration and the version disabling both use separate measured latency distributions.

SE(3)-augmented baselines. $\pi_{0.5}$+SE(3) and DynamicWAM+SE(3) receive the eight target-state tokens of Appendix A.2 from the same pose source, with the same validity flags and masking. A two-layer MLP maps the tokens to the model's width. For $\pi_{0.5}$, we append them to the prefix after the image tokens, where the action expert attends to them; for DynamicWAM, they join its kinematic descriptors. The output layer of the MLP is zero-initialized, and training follows the base model's recipe.

Ablation controls. The five controls of Sec. 4.3 change one component each of the configuration of Table S2 and keep the data, the recipe with its checkpoint selection, and the executor. *No dedicated SE(3)* removes $\mathcal{L}_P$ and the pose head, so that the target receives only the uniform coarse scene-flow supervision of every part of the scene, and *no scene-flow supervision* removes $\mathcal{L}_F$ and the flow readout; both keep the target-state tokens and all auxiliary objectives. *No target history* removes slots 0–6 of the target-state tokens, which carry the past poses with their velocity and acceleration features, and keeps slot 7 with the current pose (Appendix A.2). *No future supervision* removes $\mathcal{L}_P$, $\mathcal{L}_F$, and the three future appearance losses with their heads, and keeps the reconstruction of the current target region. *No auxiliary objectives* removes $\mathcal{L}_{\text{aux}}$, that is, the three future appearance losses and the current-target reconstruction with their heads, and keeps $\mathcal{L}_P$ and $\mathcal{L}_F$. Since every forecast head is skipped at deployment, only the no-history control changes the inputs at inference.

C. Additional Simulation Results

C.1. Per-Task Results

Table S5 breaks the held-out ConMoto results of Table 1 down by task for every method, with the paired interval of FoveaWAM against the strongest other method on each task; its last two rows give the position error at first contact e_p and the number of episodes with a contact (Appendix B.4). With the real-robot DDIM16 sampler, Diffusion Policy reaches 30.7% SR in real time and 31.0% with physics paused. OpenWAM with cache and compile disabled has no contact in some tasks, so its full 13-task e_p is undefined. [Tasks on which every method fails or a baseline outperforms FoveaWAM, and the dominant failure of FoveaWAM.]

C.2. Evaluation Protocols and Latency

Table S6 evaluates every method under three further protocols. Paused physics shows how the relative performance changes when the cost of computation is ignored, equal latency compares all methods at the p95 latency of FoveaWAM, and the original executors check whether the shared executor removes a capability that a baseline relies on. With paused physics, the difference between FoveaWAM and [name] is [±X.X] points, and under equal latency it is [±X.X] points. The original executors change the SR of the baselines by at most [X.X] points. Table S7 reports five added-latency points for nine configurations; PUMA and FoveaWAM remain pending. Fig. 4 reads this table directly and plots the selected comparisons; all methods remain in the table.

Table S5. **Per-task held-out success** (%) on ConMoto for every method of Table 1, real-time protocol. DP: Diffusion Policy. DVLA: DynamicVLA. OWAM: OpenWAM-α with official cache/compile; OW-off: both disabled. DWAM: DynamicWAM. +: with our SE(3) history tokens. σ: FoveaWAM with nominal synthetic correlated pose noise (Appendix C.5). Δ: FoveaWAM minus the strongest other method on that task, with its 95% paired interval. The last two rows give the position error at first contact e_p (mm), averaged over the 13 tasks with a contact, and the number n of episodes with a contact (Appendix B.4); –: no 13-task mean. Task codes are listed in Table S3. Shading of Δ: blue for higher SR, orange for lower, saturating at 15 points. Blue/bold and light blue rank the reported values only; pending entries are excluded.

Task	ACT	DP	$\pi_{0.5}$	$\pi_{0.5}$+	DVLA	PUMA	OWAM	OW-off	DWAM	DWAM+	σ	**Ours**	Δ [95% CI]
Harmonic grasp	49.0	31.0	51.0	**85.0**	39.0	—	41.0	1.0	50.0	51.0	—	—	—
Conveyor grasp	45.0	45.0	**96.0**	58.0	89.0	—	62.0	0.0	59.0	47.0	—	—	—
Translating grasp	14.0	33.0	59.0	38.0	**61.0**	—	53.0	0.0	53.0	15.0	—	—	—
Rotating grasp	24.0	20.0	32.0	**44.0**	26.0	—	42.0	3.0	30.0	37.0	—	—	—
Occluded grasp	30.0	28.0	37.0	51.0	**80.0**	—	34.0	0.0	44.0	36.0	—	—	—
Moving receptacle	26.0	8.0	26.0	19.0	**39.0**	—	28.0	0.0	0.0	0.0	—	—	—
Rotating receptacle	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	—	**0.0**	**0.0**	**0.0**	**0.0**	—	—	—
Wire passage	99.0	99.0	**100.0**	**100.0**	99.0	—	50.0	58.0	23.0	23.0	—	—	—
Wire passage, full	0.0	3.0	1.0	5.0	**12.0**	—	0.0	0.0	0.0	0.0	—	—	—
Ball capture	36.0	16.0	**46.0**	21.0	41.0	—	45.0	13.0	24.0	23.0	—	—	—
Ball capture, table	71.0	81.0	**100.0**	**100.0**	88.0	—	99.0	88.0	98.0	**100.0**	—	—	—
Slider capture	56.0	30.0	58.0	68.0	28.0	—	**71.0**	31.0	47.0	63.0	—	—	—
Can capture	91.0	31.0	98.0	99.0	72.0	—	**100.0**	5.0	96.0	**100.0**	—	—	—
Post-impact grasp	42.0	5.0	49.0	**51.0**	39.0	—	21.0	22.0	43.0	20.0	—	—	—
Task-balanced	41.6	30.7	**53.8**	52.8	50.9	—	46.1	15.8	40.5	36.8	—	—	—
e_p (mm)	32.2	36.2	31.2	32.5	**30.0**	—	31.7	–	31.2	37.0	—	—	–
n	1110	941	1231	1187	1203	—	1115	439	1193	970	—	—	–

Table S6. **Protocol controls** on held-out ConMoto scenes (SR, %) under the protocols of Appendix B.3; real time is the protocol of Table 1. Late: queries in real time whose time-to-action exceeds the 50 ms control period (%). Age: p95 action age in real time (ms); –: no command executed. Blue/bold and light blue rank the reported values only; pending entries are excluded.

Method	Paused	Equal latency	Original executor	Real time	Late (%)	Age p95
ACT	40.6	—	—	41.6	0.0	50
Diffusion Policy	31.0	—	—	30.7	100.0	150
$\pi_{0.5}$	66.0	—	—	**53.8**	100.0	150
$\pi_{0.5}$ + SE(3)	**74.9**	—	—	52.8	100.0	150
DynamicVLA	54.6	—	—	50.9	100.0	150
PUMA	—	—	—	—	—	—
OpenWAM-α	58.3	—	—	46.1	100.0	350
OpenWAM-α (off)	58.3	—	—	15.8	100.0	950
DynamicWAM	49.1	—	—	40.5	100.0	250
DynamicWAM + SE(3)	64.1	—	—	36.8	100.0	350
FoveaWAM	—	—	–	—	—	—

Table S7. **Success versus added latency** on 1,400 held-out ConMoto scenes per point (SR in %, real-time protocol, one training seed). The 0 and 150 ms columns equal the Native and +150 ms columns of Table 1. Blue/bold and light blue rank the reported values only; pending entries are excluded.

	Added latency (ms)				
Method	0	50	100	150	200
ACT	41.6	43.5	36.5	24.3	16.5
Diffusion Policy	30.7	26.1	21.0	15.9	14.3
$\pi_{0.5}$	**53.8**	**47.2**	**39.6**	27.8	13.6
$\pi_{0.5}$ + SE(3)	52.8	39.8	32.4	**28.2**	15.1
DynamicVLA	50.9	46.4	30.6	23.4	17.8
PUMA	—	—	—	—	—
OpenWAM-α	46.1	36.1	27.9	24.7	**22.1**
OpenWAM-α (off)	15.8	13.7	14.2	10.4	9.5
DynamicWAM	40.5	27.6	25.0	20.6	20.9
DynamicWAM + SE(3)	36.8	30.6	23.9	21.9	21.3
FoveaWAM	—	—	—	—	—

Timing. Table S8 breaks the time-to-action of FoveaWAM on one RTX 5090 down by stage, measured as in Appendix B.3; Table 1 gives the p95 latency of every method and Table S4 the parameter counts. On the robot, we report the p50, p95, and maximum action age and the fraction of all queries of the test trials that miss the 50 ms deadline, together with the peak memory.

C.3. Results on DOMINO

On DOMINO [5], we compare FoveaWAM, $\pi_{0.5}$, DynamicWAM, and PUMA on 24 single-arm Franka tasks of its clean level-1 setting. All methods use the benchmark's

Table S8. **Latency breakdown** of one query of FoveaWAM on one RTX 5090 at batch size 1 (ms, p50 / p95), and action age on the robot.

Stage	p50	p95
Pose tracking (per frame, robot)	—	—
Image encoding	—	—
World stream (once per query)	—	—
Action stream (5 Euler steps)	—	—
Time-to-action (total)	—	—
Forecast heads (training only, for reference)	—	—
Action age at execution (robot)	—	—
Queries over 50 ms (robot, %)	—	

Table S9. **Offline prediction errors.** Target-position error at 400 ms (mm) on the same held-out observations. CV and CA are analytic extrapolations from causal pose history; the learned head reads frozen full-model states.

	Const. vel.	Harmonic	Roll/slide	Impact
CV extrapolation	—	—	—	—
CA extrapolation	—	—	—	—
Learned pose head	—	—	—	—

demonstrations, scenes, and native evaluator, one training seed, and 50 episodes per task and method. We report SR and the manipulation score (MS), which scales route completion by penalties for leaving the workspace or view and for collisions (Table 2). FoveaWAM uses domain-specific normalization and reads the simulator pose of the target named by the instruction; slots without a valid pose are masked as in Appendix A.2. The RGB-based baselines do not receive this pose. [Summarize the DOMINO results.]

C.4. Final Ablations

Table S10 reports the five controls of Sec. 4.3 (Appendix B.5) with native latency and with 150 ms of added latency, each with its paired difference from the full model. The last two columns repeat the paired differences without the phase-only wire passage (`t3_phase`): on development scenes, retraining the same configuration with different seeds changed SR by up to [7.4] points, mostly on this task, whose success ranged from [0] to [71]% (Appendix B.4). The two task groups of Fig. 5 are listed in Appendix B.1. [Whether any conclusion of Sec. 4.3 changes without `t3_phase`.]

C.5. Robustness to Pose Noise

With frozen weights, we evaluate three synthetic perturbations of the target pose: nominal temporally correlated noise, twice its amplitude, and nominal noise with contiguous missing observations. The nominal process combines a fixed per-episode bias and a stationary correlated component, each with per-axis standard deviations of 5 mm and 1°, with a correlation time of 200 ms. Dropouts last 200 ms, separated by geometrically distributed valid gaps with a mean of 3.8 s, giving a long-run missing fraction of 5%. The perturbation applies to the current and past estimates before the history features are computed.

Fig. S1 evaluates the final model under these perturbations, on the same held-out scenes at zero added latency. Relative to clean poses, SR changes by [X.X], [X.X], and [X.X] points under nominal correlated noise, twice its amplitude, and missing observations.

Figure placeholder: Fig. S1 — pose noise

x: clean poses, nominal correlated noise, twice its amplitude, and nominal noise with missing observations; y: SR of FoveaWAM per task family, frozen weights, real-time protocol; paired intervals over matched scenes.

Figure S1. **Robustness to pose noise.** SR of FoveaWAM under synthetic perturbations of the target pose, relative to clean poses.

C.6. Forecast Diagnostics

Table S9 compares the learned SE(3) forecast of the final model offline with constant-velocity and constant-acceleration extrapolation from the same pose history.

Table S10. **Final ablations** on held-out ConMoto scenes (Appendix C.4). SR (%) with native latency and with 150 ms of added latency; Δ: control minus FoveaWAM (points) with its paired 95% interval. Last two columns: Δ without the phase-only wire passage (`t3_phase`), 13 tasks. Shading of Δ: blue for higher SR, orange for lower, saturating at 15 points.

	Native		+150 ms		Δ w/o `t3_phase`	
Variant	SR	Δ [95%]	SR	Δ [95%]	Native	+150 ms
FoveaWAM *	—	–	—	–	–	–
No dedicated SE(3) (uniform coarse scene flow)	—	—	—	—	—	—
No scene-flow supervision	—	—	—	—	—	—
No target history	—	—	—	—	—	—
No future supervision	—	—	—	—	—	—
No auxiliary objectives	—	—	—	—	—	—

D. Real-Robot Experiments

D.1. Platform and Tasks

The robot is a Franka arm ([model]) with [gripper model] and a fixed RealSense D435 RGB-D camera, commanded through [a timestamped Cartesian interface] at [control frequency]. The camera records RGB-D at [resolution] and [frame rate], and we calibrate its extrinsics by [procedure] before each session. FoveaWAM uses this fixed third-person view; baselines whose recipes use a wrist view also receive [wrist camera]. All policies run on one RTX 5090. The task devices share this platform and are installed in turn, one task at a time (Fig. S2). Table S11 defines the tasks. The turntable target of T3 is a block with unequal sides that must be grasped across its short sides, so that its orientation at contact matters, and the toy car of T4 runs at a fixed throttle setting.

D.2. Pose Tracking and Labels

Online tracking. Tracking is marker-free. Each object instance has one metric mesh with a fixed origin and axes, prepared once by [reference capture] and shared by all demonstrations and trials; a ball is defined by its center and radius. In the first frame, Grounding DINO [10] localizes the object named by the task, SAM 2.1 [12] segments it, and FoundationPose [16] registers its pose on the mesh. Afterwards, Cutie [3] propagates the mask in every frame, and a 6D Kalman filter predicts the pose; its position is corrected by the center of the mask's bounding box and by the median depth inside the mask, and one refinement step of FoundationPose turns the corrected pose into the estimate, which is the filter's next measurement. We run the TensorRT implementation of FoundationPose in FP16 with CUDA Graphs. An empty mask makes the estimate invalid; the previous pose is not carried forward, and [re-initialization after a loss].

Timing. The tracker runs on the control clock. At every control-period boundary, it updates the pose on the newest frame captured at or before the boundary, so each estimate is at most one camera-frame period old, and it keeps the capture time of its frame; velocities and accelerations of the target-state tokens are finite differences over these capture times. A query waits for the update of its own boundary and then runs the policy on the same GPU. While a query spans further boundaries, the tracker keeps updating at a higher GPU priority on the newest frame only, so that an update never queues behind older ones. Tracking and this contention are part of the measured latency (Table S8).

Demonstration labels. The target-state inputs of a demonstration are a causal replay of the same tracker over its recorded RGB-D at the control-period boundaries. The pose labels of the forecast are the tracked poses at the future boundaries in the same object frame, [refined offline over the episode], and the flow labels follow [3D point tracks of the recorded RGB-D, as for DROID (Appendix A.4)]. For the ball targets, the target-state tokens keep the translation, velocity, and acceleration of the center and mask the orientation and angular motion, and the pose head is supervised on translation only. Table S12 reports how often tracking was lost in the test trials.

D.3. Training and Evaluation Protocol

Training. FoveaWAM is fine-tuned per task from the DROID base (Appendix A.4). $\pi_{0.5}$, DynamicWAM, and Diffusion Policy [4] train or fine-tune on the same demonstrations with the initialization, recipe, and chunk length of Table S4, and DynamicWAM executes [with or without real-time chunking]. Demonstrations, development trials, and test trials come from separate sessions.

Trials. Each task has 20 pre-generated trial keys, which fix the object instance, its initial pose, and the device setting (Table S11). The trials are blocked by key: for each key, the four methods run consecutively in a random order, so that drift of the apparatus does not align with the methods. An operator resets the scene to the key, aligned with a reference image from the camera. A trial starts at the first usable observation, the robot may move at any time, and a trial ends at success, failure, a safety stop, or the time limit, set to about

Figure placeholder: Fig. S2 — real-robot setup

(a) Left, photo of the shared platform from behind the robot, with one task device installed: the third-person RGB-D camera, the robot, and the wrist camera if a baseline uses one, each labeled. **(b) Right, six small top-down schematics to scale, one per task (T1–T6):** the robot base frame with its axes, the camera position and viewing direction, the installed device with its footprint, the start or release region of the target and the range of its motion directions, the grasp region, and key dimensions in cm. **Rules:** real photo, unretouched; schematics from the calibrated extrinsics and measured device positions; no repetition of the task photos of Fig. 6.

Figure S2. **Real-robot setup.** (a) The shared platform. (b) Top-down layout of each task: the device, the start or release region and motion directions of the target, and the grasp region.

Table S11. **Real-robot tasks.** Target: the tracked object. Trial key: what each of the 20 test keys fixes. Speed: target speed measured from the recorded RGB-D of all test trials (m/s, mean ± s.d.). A grasp holds the object between both fingers. Each task has 100 training and 20 validation demonstrations.

Task	Target	Motion; trial key	Speed	Success	Limit (s)
T1	Block	Belt conveyor; [placement on the belt, belt speed]	—	Grasp the block and lift it by [X] cm.	[X]
T2	Open box	Belt conveyor; [box placement, belt speed]; the block starts in the gripper	—	Release the block into the box; it stays inside [until the box leaves the workspace].	[X]
T3	Block, unequal sides	Turntable; [radius, initial angle and yaw of the block, angular speed]	—	Grasp the block across its short sides and lift it by [X] cm.	[X]
T4	Toy car	Remotely driven at a fixed throttle; [start pose, heading]	—	Grasp the car and lift it by [X] cm.	[X]
T5	Tennis ball	[ball-motion mechanism, release position and motion setting]	—	Grasp the tennis ball and lift it by [X] cm.	[X]
T6	Block on cart	Cart translates down an inclined rail; [slope, release point]	—	Grasp the block and lift it by [X] cm.	[X]

Table S12. **Tracking on the real robot.** Per task, over the test trials of FoveaWAM: control-period boundaries, the fraction with an invalid estimate, the longest gap, and re-initializations.

Task	Boundaries	Invalid (%)	Longest gap (s)	Re-init.
T1	—	—	—	—
T2	—	—	—	—
T3	—	—	—	—
T4	—	—	—	—
T5	—	—	—	—
T6	—	—	—	—

three times the median duration of the task's demonstrations. A trial is repeated with the same key only if the device deviates from the key on its own, [a measured speed outside ±X% of the setting or a device fault]; everything else counts as a failure, including a tracking loss of FoveaWAM and a safety stop.

Statistics. We report successes per task and the pooled success rate with its 95% Wilson interval; with 20 trials per task, the pooled rate equals the task-balanced one. Paired differences from FoveaWAM use a bootstrap over trial keys within each task. We log the end-to-end time-to-action of every query of every method on the same RTX 5090, including tracking for FoveaWAM; Table 3 reports its p95 and Table S8 the timing on the robot.

D.4. Detailed Results and Failures

Table S13 reports the successes per task with intervals and paired differences, and the failures of each method by category. The categories, fixed before the test trials, are observable events: tracking lost (FoveaWAM only), late (the gripper closes behind the target), misaligned grasp, slip or drop after contact, collision or safety stop, and timeout. [Tasks with the largest margin and their failure categories; timing on the robot; the most frequent failures of FoveaWAM, Fig. S3.]

Table S13. **Real-robot successes and failures.** Top: successes per task (of 20 trials; tasks in Table S11); All: pooled success rate (%) with its 95% Wilson interval; Δ: paired difference from FoveaWAM (points) with a bootstrap interval over trial keys. Bottom: failures by category over all 120 trials of each method (Appendix D.4); –: not applicable. Shading of Δ: blue for higher SR, orange for lower, saturating at 15 points.

Method	T1	T2	T3	T4	T5	T6	All	Δ
$\pi_{0.5}$	—	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—	—
Diffusion Policy	—	—	—	—	—	—	—	—
FoveaWAM	—	—	—	—	—	—	—	–

	Tracking	Late	Misaligned	Slip/drop	Collision	Timeout	Failures
$\pi_{0.5}$	–	—	—	—	—	—	—
DynamicWAM	–	—	—	—	—	—	—
Diffusion Policy	–	—	—	—	—	—	—
FoveaWAM	—	—	—	—	—	—	—

Figure placeholder: Fig. S3 — failure cases of FoveaWAM

Layout: full width, two rows, one for each of the two most frequent failure categories of FoveaWAM in Table S13 (e.g., late, slip or drop); four equal 16:9 panels per row. **Panels per row:** the query frame, the first execution of its chunk, contact or the closest approach, and the end of the trial, each with its time after capture in ms from the trial log; overlays: blue tracked past poses and purple commanded gripper path; the task, the trial key, and the failure category above each row. **Rules:** real photos, unretouched; overlays from the logged data of the shown trials; captions state observable events only.

Figure S3. **Failure cases of FoveaWAM** on the real robot: [category, task, and observed event] (top) and [category, task, and observed event] (bottom).

E. Design Studies

We study the history input, future supervision, and world–action interface with 4-layer models of width 768, trained for 4 epochs (40,104 updates at global batch size 16). The earlier synchronous executor executes one command per query on 1,400 development scenes, 100 per task; the initial forecast screen reports its 140-scene selection subset and the remaining 1,260 scenes separately. Encoders, inputs, and seed counts differ between studies; comparisons are within a study. The path, auxiliary, interface, and encoder studies retain the eight-frame visual windows and GT target-motion history of their screening recipe. The final-scale, real-time ablations are reported separately in Sec. 4.3 and Appendix C.4.

E.1. History Representation

Which history helps control? With a frozen DINOv3-L encoder [13] and one training seed, we compared seven visual-history encodings against the current image alone and against the current image with the target's SE(3) history (Fig. S4a). Every condition retains the current ground-truth target pose and the robot-state history. The visual encodings include flow-based alternatives [6, 11]. The target-history condition also receives past ground-truth poses and their derived kinematics.

Target history raises SR from 53.43% to 58.86%, a gain of 5.43 points with a paired 95% scene-bootstrap interval of $[3.07, 7.86]$. None of the tested visual histories exceeds the current-image reference in point estimate. More readable history features do not necessarily improve control: for the RGB window, a frozen linear probe of the summary representation reduces the error in reading position 350 ms in the past from 10.89 to 8.46 mm, while SR falls by 6.14 points. These results support retaining explicit target motion under this training recipe.

Does the trained policy use the history? We keep the target-history checkpoint fixed, replace its past target poses by the current pose, and recompute the kinematics. Current pose, image, robot state, and timestamps remain intact. SR falls from 58.86% to 0.64% (Fig. S4b), and the first executed action changes in all 210 diagnostic windows; an identity replacement reproduces the original actions. The drop measures sensitivity to incorrect history; the training benefit is the 5.43-point comparison above. The intervals quantify scene variation conditional on these single-seed checkpoints.

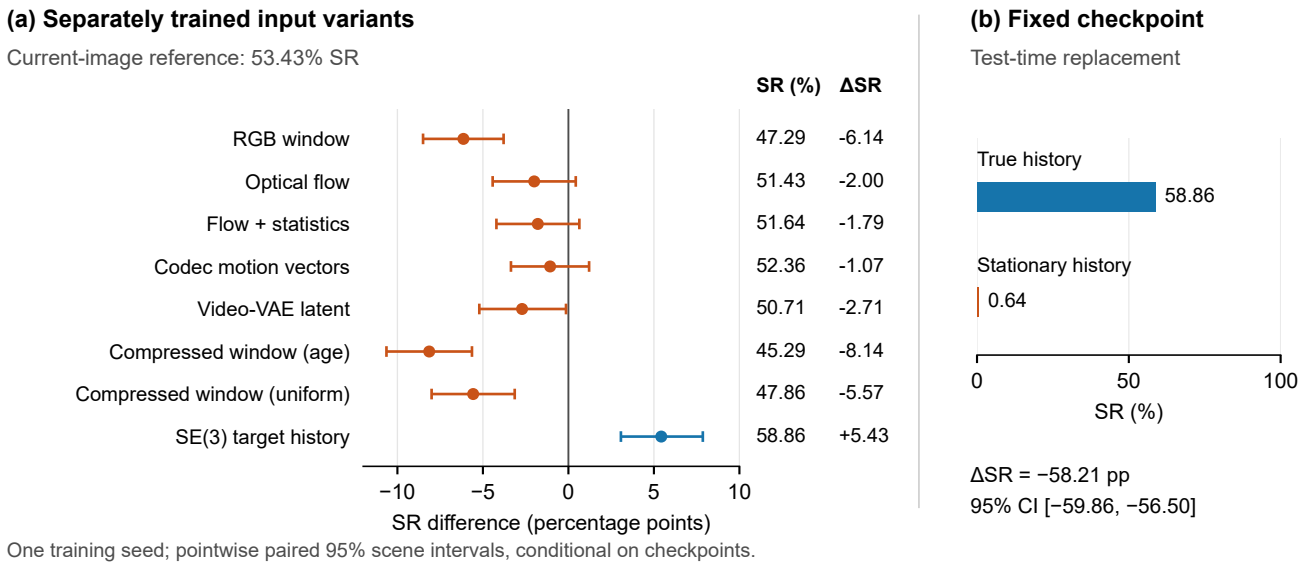


Figure S4. **History input and test-time intervention.** Frozen DINOv3-L, one training seed, 1,400 development scenes. All conditions retain the current GT target pose and robot history. (a) Separately trained variants: SR and its difference from the current-image reference. Whiskers are pointwise paired 95% scene-bootstrap intervals, conditional on trained checkpoints, without multiplicity adjustment. (b) Replacing history at the same checkpoint: 824 versus 9 successes out of 1,400. This measures sensitivity to incorrect history, separately from the training benefit in (a).

E.2. Future Representation and Supervision

Initial target screen. With frozen DINOv3-L and seed 0, we compared fifteen forecast configurations (Table S14). Geometry uses eight future horizons: target SE(3), base-frame displacements of a 14×14 patch grid, or projected 2D motion, all relative to the current state. Codec targets use current-to-future frame pairs; object DINO pools features inside the target mask. Wan2.2 VAE targets temporally compress the nine-frame clip, rather than providing one latent per horizon.

On the 1,260 remaining scenes, P07 and P09 have the highest point estimates, 64.05% and 64.13%, compared with 62.78% without future supervision (P00) and 62.38% for scene flow with SE(3) (P03). The paired comparisons do not establish an overall advantage for P07 or P09. Dense 2D displacement (P05) lowers SR by 3.57 points relative to P00 (Bonferroni-adjusted $p = 0.00844$ over 27 comparisons). These are comparisons of complete forecast configurations: P07 uses direct regression from shared features, whereas P03 uses an independent Future stream within the backbone and flow matching. Their difference therefore motivates a separate comparison of representation and prediction path.

Separating representation from prediction path. We crossed 2D and 3D targets with direct regression and the Future-stream flow-matching implementation, holding the trainable CNN, SE(3) supervision, current-target RGB reconstruction, and history inputs fixed, with three training seeds per condition (Fig. S5a). Both coordinate representations use the same source points and horizons; their scales are calibrated from training labels. Direct prediction gives higher SR than the Future-stream implementation in every seed for both 2D and 3D. Within either path, the 2D–3D difference changes sign across seeds. Across paths, the loss, motion weighting, and visibility supervision also change, so the result favors the complete direct-prediction recipe.

A further comparison fixes the direct path, frozen DINOv3-L, and all three future appearance targets (Fig. S5b). The 2D–3D difference is $+2.00$, -1.50, and $+1.00$ points across seeds, with a mean of $+0.50$. We retain 3D displacement to express scene motion in the robot's metric frame; these development studies do not establish a control-performance advantage of 3D over 2D.

Selecting auxiliary supervision. With frozen MAE-B and seed 1, we keep direct 3D flow, SE(3), and current-target RGB reconstruction, and compare no future appearance target, each target alone, and all three together (Table S15). DINO and dynamic RGB alone improve on the no-auxiliary reference, depth alone lowers SR, and the combination has the highest SR, 71.07%. The single-target rows measure the effect of adding that target to the reference; they do not isolate its marginal contribution when the other two are present. This screen motivates retaining the combination with MAE-B. Auxiliary benefits depend on the encoder: in

Table S14. **Initial forecast-configuration screen.** Frozen DINOv3-L, seed 0. The 140-scene selection subset and the remaining 1,260 development scenes are disjoint. Rows are grouped by the changed target; prediction paths also differ, as described in Appendix E.2. A dash means that target is absent. Blue/bold and light blue mark the best and second point estimates within each outcome column, not statistical significance.

ID	Scene target	Object target	Additional target	Success / 140	SR_{1260} (%)
Basic supervision controls					
P00	–	–	–	88	62.78
P01	3D scene flow	–	–	**95**	62.06
P02	–	SE(3)	–	92	62.62
P03	3D scene flow	SE(3)	–	93	62.38
Scene-target alternatives to P03					
P04	Video-VAE latent	SE(3)	–	92	62.14
P05	Dense 2D displacement	SE(3)	–	87	59.21
P06	Codec motion vectors	SE(3)	–	86	63.10
P07	Sparse 2D tracks	SE(3)	–	90	64.05
Object-target alternatives to P03					
P08	3D scene flow	Object DINO	–	89	63.02
P09	3D scene flow	Target 2D tracks	–	88	**64.13**
Additional targets on P03					
A_α	3D scene flow	SE(3)	Motion magnitude	89	61.75
P12	3D scene flow	SE(3)	Depth	90	62.78
P13	3D scene flow	SE(3)	DINO	92	62.54
P14	3D scene flow	SE(3)	Dynamic RGB	89	62.70
P15	3D scene flow	SE(3)	Depth, DINO, dynamic RGB	94	63.65

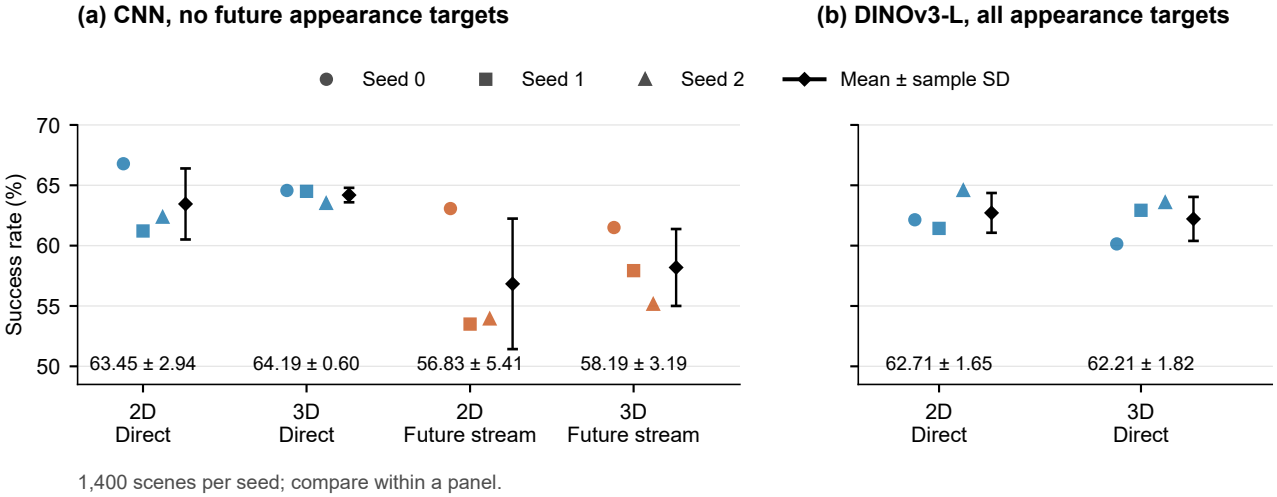


Figure S5. **Representation and prediction path.** Colored markers show three training seeds, each evaluated on 1,400 development scenes; black diamonds and whiskers show their mean and sample standard deviation, also printed below. Both panels retain SE(3) and current-target RGB supervision. (a) CNN without future appearance targets: direct regression versus the Future-stream flow-matching recipe. (b) Frozen DINOv3-L, direct regression, and all three appearance targets. Compare within each panel; the seed spread is not a confidence interval.

the three-seed CNN study, all three targets average 62.86%, versus 63.95% with none. The final-model no-future control in Sec. 4.3 tests the full forecast supervision set, including the geometric targets.

E.3. World–Action Information Flow

Interfaces and controls. With frozen MAE-B and shared attention with separate world/action feed-forward networks, we distinguish observed conditions C, future-query hidden

Table S15. **Future appearance supervision.** Frozen MAE-B, seed 1, 1,400 scenes; direct 3D flow, SE(3), and current-target RGB reconstruction stay enabled. Δ: SR difference from the first row (points). ✓: enabled; –: disabled. Blue/bold and light blue mark the highest and second SR point estimates. Δ shading: blue for higher SR, orange for lower, saturating at 15 points; it does not encode significance.

Depth	DINO	Dynamic RGB	SR (%)	Δ
–	–	–	63.86	ref.
✓	–	–	61.93	−1.93
–	✓	–	67.57	+3.71
–	–	✓	66.14	+2.29
✓	✓	✓	**71.07**	+7.21

Table S16. **World–action information flow.** Frozen MAE-B, three training seeds, 1,400 development scenes per seed. W: future-query hidden states; A: action states; Y: decoded geometry. Actions always read observed conditions C. R1–R4 form the two-edge comparison. SR is the seed mean ± sample s.d. (%); Δ_{R1} is the mean ± sample s.d. of same-seed differences from R1 (points). Blue/bold and light blue mark the highest and second SR point estimates. Δ shading: blue for higher SR, orange for lower, saturating at 15 points; it does not encode significance.

ID	Future losses	$W \to A$	$A \to W$	$Y \to A$	SR (%)	Δ_{R1}
R0	–	–	–	–	60.45 ± 3.57	-3.14 ± 5.75
R1	✓	–	–	–	63.60 ± 2.34	ref.
R2	✓	✓	–	–	62.52 ± 1.97	-1.07 ± 4.28
R3	✓	–	✓	–	$\mathbf{64.21 \pm 2.13}$	$+0.62 \pm 0.84$
R4	✓	✓	✓	–	63.00 ± 4.19	-0.60 ± 6.52
R5	✓	–	–	✓	63.95 ± 1.91	$+0.36 \pm 3.45$

states W, and decoded predictions Y. Actions always read C. One object query and one query per patch read C and one another to predict SE(3) and scene flow. R1–R4 keep the modules, losses, and initialization procedure fixed and toggle the two attention edges $W \to A$ and $A \to W$ (Table S16). Their recorded joint evaluation recomputes W at each Euler step. R0 keeps the same modules but disables all future losses; current RGB reconstruction remains. R5 instead decodes Y once per query and re-encodes the trajectory, flow, and visibility into action conditions with a 0.37M-parameter geometry encoder. It uses predicted geometry during both training and inference, with action gradients flowing through that interface.

Supervision and action conditioning. R1 improves on R0 by 3.14 points on average, with seed-wise differences of −1.29, +9.64, and +1.07. Adding hidden-state access (R2) or decoded geometry (R5) changes SR relative to R1 by −1.07 and +0.36 points, respectively, with seed-wise difference standard deviations of 4.28 and 3.45 points. Neither produces a consistent gain across seeds. R3 has the highest mean, but its 0.62-point gain over R1 becomes −0.03 points outside the phase-only wire task; all fourteen tasks remain in the primary metric. The main effects of the two edges and their interaction also change sign across seeds.

Relation to the deployed model. We retain future supervision without a forecast-to-action interface. R1 uses independent future queries; FoveaWAM reads forecasts directly from the existing scene and summary tokens (Sec. 3.3). These distinct architectures share the supervision-only use of forecasts. In FoveaWAM, actions still read the observed world representation, while forecast heads are skipped at deployment.

E.4. Encoder and Parameter Sharing

Table S17. **Encoder and parameter sharing.** Three seeds, 1,400 scenes per seed; direct 3D, SE(3), and all appearance targets. SR: seed mean ± sample s.d. (%). Panel (b) fixes MAE-B, reusing (a)'s shared-attention runs. Params: trainable backbone; ⋆: selected. Blue/bold and light blue mark the best and second point estimates within each panel and outcome column, not statistical significance.

Configuration	Status / params	SR (%)
(a) Observation encoder		
CNN	trainable	62.86 ± 2.64
DINOv3 ViT-S/16	frozen	61.69 ± 2.00
DINOv3 ViT-L/16	frozen	62.21 ± 1.82
MAE ViT-B/16⋆	frozen	$\mathbf{66.76 \pm 3.84}$
(b) World/action parameter sharing		
Shared attention⋆	47.266M	$\mathbf{66.76 \pm 3.84}$
Separate parameters	47.272M	62.86 ± 2.98

Observation encoder. We compare a trainable CNN with frozen DINOv3-S, DINOv3-L [13], and MAE-B [7], keeping direct 3D prediction, SE(3), and the three appearance targets (Table S17a). Each uses three seeds and the same 8-GPU training recipe and global batch size. MAE-B has the highest mean SR, 66.76%, and is selected for the subsequent interface and backbone studies.

Parameter sharing. With MAE-B fixed, we compare shared attention and stream-specific feed-forward networks against separate world/action attention and feed-forward parameters (Table S17b). Both keep the same information flow and about 47.3M trainable backbone parameters; the separate variant uses a smaller feed-forward width to match this count. Shared attention averages 66.76% SR, against 62.86% for separate parameters. Their mean gap falls to 0.39 points when both wire tasks are excluded, so the difference is concentrated in those tasks. We retain shared attention based on this aggregate comparison.

References

[1] Kevin Black, Noah Brown, James Darpinian, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, pages 17–40, 2025. 6

[2] Kevin Black, Manuel Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems*, pages 33383–33407, 2025. 5

[3] Ho Kei Cheng, Seoung Wug Oh, Brian Price, Joon-Young Lee, and Alexander Schwing. Putting the object back into video object segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. 9

[4] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin C. M. Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 6, 9

[5] Heng Fang, Shangru Li, Shuhan Wang, Xuanyang Xi, Dingkang Liang, and Xiang Bai. Towards generalizable robotic manipulation in dynamic environments. In *European Conference on Computer Vision*, pages 314–333, 2026. 6, 7

[6] Gunnar Farnebäck. Two-frame motion estimation based on polynomial expansion. In *Proceedings of the Scandinavian Conference on Image Analysis*, pages 363–370, 2003. 11

[7] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022. 1, 14

[8] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20765–20779, 2026. 3

[9] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024. 3

[10] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *European Conference on Computer Vision*, 2024. 9

[11] Yunfan Lou, Hewen Gao, Xiyu Zhu, Zhuoran Qiao, Xuan Han, Yifan Yang, Yifan Ye, Boxian Yao, and Zhibo Pang. DynamicWAM: Dual-path motion conditioning for world-action models in dynamic manipulation. *arXiv preprint arXiv:2608.00793*, 2026. 6, 11

[12] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations*, pages 28085–28128, 2025. 9

[13] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. 2, 11, 14

[14] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. 3

[15] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An open, modular exploration towards systematic world-action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026. 6

[16] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. FoundationPose: Unified 6D pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17868–17879, 2024. 9

[17] Haozhe Xie, Beichen Wen, Jiarui Zheng, Zhaoxi Chen, Fangzhou Hong, Haiwen Diao, and Ziwei Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation. In *Advances in Neural Information Processing Systems*, 2026. Accepted for publication. 6

[18] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. 6

[19] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. 3